Build Assignment: Docker Load Balancer with MCP Controls

Oct 7, 2026

Route traffic between two Docker containers, A and B, and control it from a web dashboard and an MCP client.

Deadline: Oct 14, 2026 Wednesday

What to build

1. Containers A and B

- Each response says which container handled it
- A readiness (health) check
- An optional artificial delay, used to demo draining
- A way to force the container unhealthy, used to demo a failed rollout

2. Controller and load balancer

Case	Required behaviour
Traffic split	A/B percentages that add up to 100
Routing	New requests go only to healthy containers that are not draining
One container unhealthy	Temporarily send all traffic to the other; show the effective split and log it
Neither container healthy	Return a service-unavailable response
Invalid split	Reject any split that sends traffic to a stopped, unhealthy, or draining container
Stop	Remove from routing → let in-flight requests finish → stop
Stopping the last healthy container	Reject
Drain timeout	Report an error; keep the container running but send it no new requests

Case	Required behaviour
Safe rollout	Start target → wait until healthy → move traffic → drain old → stop old
Rollout timeout	Abort; traffic stays on the old container
During a rollout	Reject any conflicting change

3. Traffic generator

- Web UI or CLI to set requests per minute and start/stop traffic
- Sends all requests through the load balancer
- Shows total, successful, and failed requests; requests served by A and by B; the current split

4. Live dashboard

- Status of A and B: running, healthy, draining, or stopped
- Controls: change split, start/stop a container, safe rollout
- Current split and request count per container
- Event log: time, action, source, result

5. MCP controls

Capability	What it does
Status	Returns state, health, split, counts, and errors
Traffic split	Changes the split, with validation
Start container	Starts the container, then waits until it is healthy or times out
Stop container	Drains the container, then stops it
Safe rollout	Moves traffic from one container to the other
Recent logs	Returns recent controller events

- MCP and the dashboard must use the same controller logic
- Log every MCP action with source `mcp`, including rejected ones
- Invalid operations must return a clear error
- You choose the endpoint paths, MCP tool names, language, and framework

Demo video (3–5 minutes)

1. Traffic running on A
2. An MCP client checks status, then starts a safe rollout from A to B
3. The dashboard shows each rollout step, with source `mcp`
4. Traffic now on B, with zero failed requests
5. An in-flight request finishes on A before A stops
6. A failed rollout: the target never becomes healthy, and the original container keeps serving
7. An invalid operation returns a clear error

Submit

- Code (Git repo or ZIP) with Docker Compose; `docker compose up` must start everything
- README covering setup, architecture, MCP client configuration, your endpoint paths and tool names, and your health check and drain timeouts
- Link to the demo video
- Which AI tools you used, and what you used each one for

Evaluation

Area	Weight
Controller API, validation, and state management	30%
Docker setup and health checks	20%
Traffic routing, draining, and rollout safety	20%
MCP integration	15%
Logs, dashboard, and demo	10%
Explaining and modifying your code	5%

Everything runs locally. Keep A and B simple.